



SolarAgents: EloT Workshop Proposal and Implementation Plan

面向 The 9th International Workshop on Embodied Intelligence of Things (EloT 2026)

定位

建议将当前 SolarChain 从“区块链支持的城市太阳能交易平台”重构为：

SolarAgents: a physics-grounded embodied IoT agent coordination testbed for verifiable urban solar energy systems.

区块链仍然保留，但不再作为论文主角。它应被重新定位为 **verifiable actuation / settlement layer**：当 embodied solar agents 的物理报告通过验证和 planner 审核后，链上合约负责登记、激励、结算和可追溯记录。论文主线转为：

physical perception -> physics-bounded reasoning -> agent action
-> multi-agent coordination -> verifiable settlement -> feedback/adaptation

这比当前的：

CSV dataset -> frontend review -> smart contract registration -> liquidity chart

更贴近 EloT 的核心主题。

1. Workshop 主题对齐

EloT 2026 官网明确说明 workshop 关注 physically grounded, distributed, and evolving intelligence in ubiquitous computing。主题包括 physical-to-digital mapping、opportunistic/context-aware interaction、distributed sensing and actuation、safety/verifiability、benchmarks/datasets，并且 application areas 中明确包括 smart energy systems。

官方主题与本项目的最佳对齐关系如下：

EIoT 2026 topic	当前 SolarSave 已有基础	需要新增的内容	论文中应强调
Physically grounded perception and reasoning	P_max_W ` P_reported_W ` pvlib/ Open-Meteo 数据融合 ` FDIA verification	将 DER record 封装成 SolarAgent 的 perception/ reasoning/action loop	太阳能 DER 作为 physics sensor context + physics embodied agent
Physical-to-digital mapping under resource, energy, latency constraints	FastAPI predictor ` CSV benchmark ` planner queue	latency/ throughput/ system overhead benchmark	轻量 physics-bound ver EIoT pipeline 中运行
Opportunistic/ context-aware interaction among embodied agents	planner review ` factory demand ` market loop	event-driven coordination loop: solar/factory/ planner/market agents	多 agent 协调不是抽象叙事 ’而是
Distributed sensing and actuation for CPS	5 城市 ` 50 PV nodes ` factory demand ` smart contracts	event log + coordination metrics	城市 DER 网络作为 distributed physical energy agents
Safety, verifiability, ethics of autonomous/ decentralized	human-in-the-loop UI ` on-chain events	persistent audit log ` hash chain ` attack taxonomy	物理约束 + planner + security 三层可验证性

EIoT 2026 topic	当前 SolarSave 已有基础	需要新增的内容	论文中应强调
systems			
Evaluation frameworks, benchmarks, datasets for embodied IoT	1,200 records `FDIA labels `market CSV	attack taxonomy `ratio sweep `agent adaptation `scale benchmark	从 demo dataset 升级为 benchmark suite

这意味着投稿时不要说“我们做了 blockchain energy market” ’而要说：

We instantiate urban photovoltaic DERs as lightweight embodied IoT agents whose physical body, sensing context, reporting action, verification outcome, market actuation, and feedback history form a reproducible EIoT coordination loop.

2. 文献综述脉络

2.1 从 CPD/PICASSO 到 EIoT

EIoT 2026 是 CPD 与 PICASSO workshop series 的延续 °CPD 2023 的主题是 Combining Physical and Data-Driven Knowledge in Ubiquitous Computing ’强调在 ubiquitous computing systems 中融合 physical knowledge `domain knowledge `heuristics 和 analytic models ’应用方向包括 smart cities 和 smart energy systems °

EIoT 2026 在此基础上进一步拓展：从 physical-knowledge-informed sensing 扩展到 embodied agents `distributed coordination 和 self-evolving systems °因此 ’SolarChain 原有的 physics-bounded verification 是一个很好的入口 ’但还不够 °它需要从“物理规则验证数据”升级为“物理 agent 在闭环系统中感知 `推理 `行动 `协调和演化” °

可引用来源：

- EIoT 2026 official site: <https://eiot-workshop.github.io/>
- CPD 2023 official site: <https://ubicomp-cpd.com/2023.html>
- PICASSO 2025 official site: <https://picasso-2025.github.io/2025.htm>

2.2 EIoT / Physical AI Agents

EIoT 概念论文将 EIoT 描述为 distributed, physically grounded intelligent systems ’ 并强调 Enacted ` Engaged ` Evolutionary 三个维度 ˆ

- **Enacted:** devices are transformed into embodied agents.
- **Engaged:** agents interact under physical and contextual constraints.
- **Evolutionary:** intelligence adapts and self-organizes through experience.

与之相邻的 Physical AI Agents 文献也强调 ˆIoT 过去主要是 sense and report ’ 而 physical AI agents 应该 perceive, reason, act, cooperate, and intervene ° 对于本项目而言 ’ 太阳能 DER 节点并不需要变成通用智能体 ˆ 它们只需要成为轻量 ` 物理接地 ` 可验证 ` 可反馈的 energy agents °

可引用来源 ˆ

- Liu et al., "EIoT: Embodied Intelligence of Things", Tsinghua Science and Technology, 2026. <https://www.sciopen.com/article/10.26599/TST.2025.9010127>
- Morabito and Tatipamula, "The Internet of Physical AI Agents", arXiv 2026. <https://arxiv.org/abs/2603.15900>

2.3 AIoT 与边缘智能

AIoT survey 将 AIoT 文献组织为 sensing ` computing ` networking/communication 和 domain-specific systems ° SolarAgents 可以定位为 AIoT/EIoT 在 smart energy domain 的轻量系统实例 ˆ 并不追求大模型 ’ 而是用物理模型 ` agent 状态 ` 规则推理和可验证结算构成小而完整的闭环 °

可引用来源 ˆ

- Siam et al., "Artificial Intelligence of Things: A Survey", ACM Transactions on Sensor Networks. <https://dl.acm.org/doi/10.1145/3690639>

2.4 Physics-informed anomaly detection

智能电网和电力系统中的 FDIA 研究已经证明 ’ 纯数据驱动模型存在泛化 ` 解释性和物理一致性问题 ° Physics-informed approaches 通过把物理法则或约束放进模型或检测逻辑 ’ 提升数据稀缺 ` 噪声 ` 跨场景迁移下的鲁棒性 °

SolarAgents 的优势是：PV 输出上界可以由太阳辐照、温度、面板面积、效率和温度系数给出，因此天然适合 physical-bound verification。需要避免的风险是：不要宣称所有 FDIA 都能被单一上界抓住。physically plausible fraud、sensor drift、replay attack 需要 temporal consistency、neighbor consistency 或 agent memory。

可引用来源：

- Raissi et al., "Physics-informed neural networks", Journal of Computational Physics, 2019. <https://ui.adsabs.harvard.edu/abs/2019JCoPh.378..686R/abstract>
- Zideh and Solanki, "Physics-Informed Convolutional Autoencoder for Cyber Anomaly Detection in Power Distribution Grids", arXiv 2023. <https://arxiv.org/html/2312.04758v1>
- Zraggen et al., "Physics Informed Deep Learning for Tracker Fault Detection in Photovoltaic Power Plants", PHM Society, 2022. <https://papers.phmsociety.org/index.php/phmconf/article/view/3235>

2.5 Smart-grid FDIA and P2P energy trading

FDIA 是 smart grid 的核心安全问题之一，尤其在 P2P energy trading 中，虚假供给/需求报告会直接影响激励、市场流动性和结算公平。SolarAgents 应该明确把攻击模型分成：

- physically impossible attacks: 物理上界可以稳定抓住；
- physically plausible attacks: 需要 temporal/neighbor/market feedback；
- economic manipulation attacks: 需要市场层实验衡量影响。

可引用来源：

- Mohammadi et al., "Detecting False Data Injection Attacks in Peer to Peer Energy Trading Using Machine Learning", IEEE TDSC. <https://www.computer.org/csdl/journal/tq/2022/05/09483661/1vcJui7s3Ys>
- Reda et al., "Comprehensive survey and taxonomies of false data injection attacks in smart grids", Renewable and Sustainable Energy Reviews, 2022. <https://ideas.repec.org/a/eee/rensus/v163y2022ics1364032122003306.html>

3. 现有代码基础与问题

3.1 已有优势

当前仓库已经具备投稿 EIoT 的可用基础：

已有模块	文件	可转化为 EIoT 的意义
多城市 PV benchmark dataset	Simulator/data/generate_multicity_datasets.py	physical-to-digital mapping, smart energy benchmark
P_max_W / P_reported_W verification	Simulator/data/datasets/spatiotemporal_generation.csv	physics-grounded perception/reasoning
planner review UI	client/src/components/MapSection.jsx	human-agent collaboration / planner-in-the-loop
on-chain panel/factory registry	smart_contract/contracts/SolarPanels.sol , Factory.sol	physical-to-digital twin registry
energy settlement	smart_contract/contracts/EnergyExchange.sol	verifiable actuation / settlement
market update loop	Simulator/main.py	closed-loop coordination 的雏形

3.2 当前主要缺口

缺口	当前表现	EIoT 风险
没有 agent abstraction	DER node 是 CSV row / map marker / panel struct	容易被看作普通 IoT/ blockchain demo
没有 event-driven closed loop	<code>market_loop</code> 只是定时更新合约	不足以支撑 sense-reason-act-coordinate
1:3 split hard-coded	合约中 25/75 固定 '数据生成中 liquidity 公式固定	审稿人会质疑机制是预设结果
FDIA taxonomy 太简单	主要是 night / above-bound ' 且 <code>0.42 * Pmax</code> 被直接标 rejected	不能诚实展示物理上界的边界
audit 不持久	前端 <code>auditEvents</code> 只在 session 内保存	不够 verifiability / accountability
没有 evolutionary experiment	无 residual memory / trust update / calibration	不能回应 EIoT 的 evolutionary 维度
工程安全卫生不足	hardcoded API key, CORS *	与 trustworthy EIoT 叙事冲突

3.3 Anti-wrapper 标准：什么才不算 CSV 换皮

这一点必须写进项目设计 ' 也应该隐含体现在论文叙事里 ° 否则审稿人很容易认为 SolarAgents 只是把普通 CSV 仿真包装成 agent °

不能算真正 EIoT agent layer 的情况：

1. 每行 CSV 独立计算，没有跨小时状态。
 2. SolarAgent 只是读取 row 并返回 Pmax/report/status。
 3. planner/market/factory 只是后处理统计，没有影响 agent 下一步行为。
 4. attack detection 只是离线分类，没有事件链和反馈。
 5. on-chain settlement 与 agent 决策完全脱节。
 6. 前端仍只展示静态 CSV queue，没有展示 agent event trace。

要达到 EIoT workshop 级别 ' 至少需要满足以下验收标准：

Criteria	必须实现的证据
Persistent embodied state	每个 SolarAgent 跨小时保存 trust_score , residual_history , calibration_factor , participation_state
Closed perception-action loop	weather_update -> perceive -> report_generation -> verification -> feedback 写入 event log
Coordination dependency	factory demand / market slippage / planner rejection 会改变 agent 后续参与或 trust
Adaptation evidence	static verifier 与 adaptive agent 在 drift/replay/near-bound attack 下有对比实验
Verifiable actuation	approved agent report 可以进入现有 FastAPI/frontend/on-chain settlement path
Persistent audit trail	event hash chain 或 audit CSV/SQLite 记录 machine/planner/on-chain action

论文里应明确声明：

We do not claim general-purpose AI agency. Instead, we implement lightweight embodied IoT agents with persistent physical state, rule-based physical reasoning, planner/market feedback, and measurable adaptation over repeated physical-energy reporting cycles.

4. 核心改造方案：新增 Agent Layer

4.1 新增目录

建议新增：


```
Simulator/agents/  
  __init__.py  
  types.py  
  environment.py  
  event_bus.py  
  solar_agent.py  
  factory_agent.py  
  planner_agent.py  
  market_agent.py  
  coordination_loop.py  
  api_bridge.py
```

```
Simulator/experiments/  
  eiot_agent_run.py  
  ratio_sweep.py  
  attack_taxonomy.py  
  baseline_detectors.py  
  adaptive_verification.py  
  system_overhead.py  
  scale_benchmark.py
```

输出：

```
Simulator/data/experiments/  
  eiot_event_log.csv  
  audit_logs.csv  
  ratio_sweep_results.csv  
  attack_taxonomy_results.csv  
  baseline_comparison_results.csv  
  adaptive_agent_results.csv  
  system_overhead_results.csv  
  scale_benchmark_results.csv  
  connected_system_trace.csv
```

4.2 Agent 类型

SolarAgent

`SolarAgent` 是论文主角 °它将每个 DER node 从 CSV row 升级成 embodied IoT agent °

状态 :

```
body:
  node_id
  city
  latitude
  longitude
  panel_area_m2
  efficiency
  temp_coefficient

perception:
  irradiance_Wm2
  air_temp_C
  sensor_noise
  weather_confidence

belief:
  p_max_W
  expected_report_W
  residual_history
  trust_score
  calibration_factor
  neighbor_consistency_score

action:
  report_generation()
  request_registration()
  join_market()

feedback:
  verification_result
  planner_action
  reward_energy
  audit_flag
  market_slippage
```

最小 Python 结构：

```

@dataclass
class SolarAgent:
    agent_id: str
    city: str
    latitude: float
    longitude: float
    panel_area_m2: float
    efficiency: float
    temp_coefficient: float
    trust_score: float = 1.0
    calibration_factor: float = 1.0
    residual_history: list[float] = field(default_factory=list)

    def perceive(self, weather_event: WeatherEvent) -> Perception:
        ...

    def estimate_pmax(self, perception: Perception) -> float:
        ...

    def report_generation(self, perception: Perception, attack: AttackConfig | None) -> So
        ...

    def receive_feedback(self, feedback: AgentFeedback) -> None:
        ...

```

`receive_feedback()` 不需要复杂 AI °第一版用规则即可：

```

if planner_action == rejected:
    trust_score -= 0.15
elif verification_result == verified:
    trust_score += 0.02

if residual_history shows persistent positive residual:
    calibration_factor *= 0.995

trust_score is clipped to [0, 1]
calibration_factor is clipped to [0.85, 1.10]

```

这样就能体现 evolutionary ’但不会引入 GPT API 或不可复现复杂度 °

FactoryAgent

FactoryAgent 表示城市需求侧 agent °

状态：

```
factory_id
city
latitude
longitude
base_demand_W
demand_profile
risk_tolerance
unmet_demand_history
```

行为：

```
observe_market()
decide_purchase()
consume_energy()
receive_feedback()
```

规则示例：

```
if slippage > threshold:
    purchase = demand * reduced_factor
else:
    purchase = min(demand, available_liquidity)
```

意义：把系统从“供应侧验真”扩展为 supply-demand coordination °

PlannerAgent

PlannerAgent 对应现在前端中的 human-in-the-loop planner ’但用于离线实验时采用 rule-based policy °

策略：

```
machine rejected -> reject
near-bound + low trust -> escalate/reject
near-bound + high trust -> approve with audit flag
verified + normal residual -> approve
within-bound suspected fraud -> escalate
```

输出：

```
planner_action: approve / reject / escalate
audit_reason
human_review_required
```

意义：让 planner review 不只是 UI，而成为可评估的 human-agent collaboration 模块。

MarketAgent

MarketAgent 接收 verified supply 和 factory demand，执行 reward/liquidity split。

参数：

```
reward_share
liquidity_share = 1 - reward_share
reserve_buffer
slippage_epsilon
producer_exit_threshold
```

计算：

```
reward_energy = verified_energy * reward_share
liquidity_energy = verified_energy * liquidity_share + reserve_buffer
fulfilled_demand = min(factory_demand, liquidity_energy)
unmet_demand = max(0, factory_demand - liquidity_energy)
slippage = trade_size / (liquidity_energy + epsilon)
```

意义：把 1:3 从固定叙事变成可实验机制。

4.3 Environment 与 EventBus

Environment 负责生成 weather events :

```
timestamp
hour
city
irradiance_Wm2
air_temp_C
cloud_factor
weather_noise
```

EventBus 负责持久化所有事件 :

```
event_id
timestamp
agent_id
agent_type
event_type
physical_state
reported_state
verification_result
planner_action
on_chain_action
market_feedback
latency_ms
record_hash
previous_hash
```

这会生成 `eiots_event_log.csv` 成为论文里的 benchmark artifact °

4.4 闭环流程

每小时执行 :

- 1. Environment emits weather_update(city, hour)
- 2. SolarAgent perceives irradiance/temp/noise
- 3. SolarAgent estimates Pmax
- 4. SolarAgent reports generation, optionally under attack
- 5. Machine verifier checks physics/temporal/neighbor consistency
- 6. PlannerAgent approves/rejects/escalates
- 7. MarketAgent aggregates verified supply
- 8. FactoryAgent observes market and purchases energy
- 9. MarketAgent computes slippage, unmet demand, reward
- 10. SolarAgent and FactoryAgent receive feedback
- 11. EventBus writes audit and coordination events

论文中可以画成一张 EloT loop figure °

4.5 Connected System Mode ：避免只停留在离线仿真

为了把贴合度从“不错的离线 benchmark”提高到“更像 EloT system paper” ’建议做一个 **Connected System Mode** °它不要求真实物理面板部署 ’但要求 agent simulation 与当前 FastAPI `React frontend 和 Solidity settlement path 连接起来 °

系统应支持两种运行模式 ：

Mode	用途	是否需要本地区块链
offline_benchmark	大规模实验 `ratio sweep `FDIA taxonomy `adaptive verification	不需要
connected_demo	展示 agent event trace `planner review `approved report 到合约 settlement 的完整路径	需要 Hardhat local node + MetaMask 或 simulator private key

4.5.1 FastAPI integration

当前 Simulator/main.py 已经提供 /run_model/ 和后台 market_loop °建议新增 agent endpoints ：


```
GET /agents/status
POST /agents/step
POST /agents/run_episode
GET /agents/events
GET /agents/audit
GET /agents/market_summary
POST /agents/settle_verified_step
```

职责：

```
/agents/step:
    执行一个 weather -> solar report -> verification -> planner -> market feedback step

/agents/run_episode:
    执行 24h 或 168h agent episode, 输出 event log

/agents/events:
    返回最近 N 条 event trace, 给前端展示

/agents/settle_verified_step:
    将当前 step 中 approved 的 verified supply 汇总后调用 EnergyExchange.updateMarketStep
```

建议新增 `Simulator/agents/api_bridge.py` 用于把 agent 内部对象转换成 FastAPI JSON：

```
AgentStatedTO
AgentEventDTO
PlannerDecisionDTO
MarketSummaryDTO
SettlementRequestDTO
```

这样前端和论文都能展示“agent 系统正在运行”而不是只展示静态 CSV。

4.5.2 Frontend integration

当前前端 `MapSection.jsx` 已经有 Candidate DER Queue `planner evidence panel 和 session 内 audit events` 建议新增：

```
client/src/components/AgentTrace.jsx
client/src/components/AgentCoordination.jsx
client/src/utils/agentApi.js
```

前端展示：

1. Agent Event Trace:
weather_update / solar_report / verification_result / planner_decision / market_update ,
2. Agent State Panel:
trust_score, calibration_factor, residual_history sparkline, current participation_sta
3. Connected Settlement Panel:
approved supply, demand, reward/liquidity split, tx_hash if settled
4. Anti-wrapper Evidence:
show previous trust -> feedback -> updated trust for the selected solar agent

这可以直接证明 agent 有跨时间状态和反馈 ’不是普通 CSV row °

4.5.3 On-chain integration

Connected mode 应复用当前链上合约 ’而不是做完全离线市场表格 °最小路径：

```
SolarAgent approved reports
-> MarketAgent aggregates userEnergy/totalEnergy/demandEnergy
-> FastAPI /agents/settle_verified_step
-> EnergyExchange.updateMarketStep(users, userEnergy, totalEnergy, demandEnergy)
-> tx_hash written to eiot_event_log.csv and audit_logs.csv
```

如果合约完成 configurable split ’则 connected mode 还应：

```
setRewardRatioBps(2500)
run settlement
read globalSupplyEnergy and personalRewardWei
write on_chain_feedback event
```

论文中可以把它表述为：

The blockchain layer is not used to define agency. It is used as a verifiable actuation substrate: approved agent actions are committed to a local EVM ledger, and transaction hashes are linked back into the agent event log.

4.5.4 Connected trace 输出

新增：

Simulator/data/experiments/connected_system_trace.csv

字段：

episode_id
step
timestamp
agent_id
agent_type
event_type
previous_trust
updated_trust
p_max_W
p_reported_W
verification_result
planner_action
reward_share
liquidity_share
market_liquidity_W
factory_demand_W
fulfilled_demand_W
on_chain_action
tx_hash
latency_ms
record_hash
previous_hash

这张表是“不是 CSV 换皮”的核心证据。普通 CSV 仿真只有输入和输出；connected trace 记录的是跨 agent、跨时间、跨系统层的状态转移。

5. 合约和市场机制改造

5.1 EnergyExchange.sol

当前问题：25 和 75 在合约中硬编码。

建议改成：

```
uint256 public constant BPS = 10000;
uint256 public rewardRatioBps = 2500;

event SplitRatioUpdated(uint256 rewardRatioBps, uint256 liquidityRatioBps);

function setRewardRatioBps(uint256 ratio) external onlyOwner {
    require(ratio <= BPS, "Invalid ratio");
    rewardRatioBps = ratio;
    emit SplitRatioUpdated(ratio, BPS - ratio);
}
```

updateMarketStep() 中：

```
uint256 rewardEnergy = (userEnergy[i] * rewardRatioBps) / BPS;
uint256 supplyEnergy = (totalEnergy * (BPS - rewardRatioBps)) / BPS;
```

同时加 Hardhat tests：

```
rewardRatioBps default is 2500
owner can set ratio
non-owner cannot set ratio
ratio > 10000 reverts
market step uses configured ratio
```

5.2 数据生成器

当前 make_market_liquidity() 使用：

```
solarchain_liquidity_MW = total_verified_MW * 0.92 + 0.018
baseline_liquidity_MW = total_verified_MW * 0.61 + 0.008
```

建议改为机制驱动：

```
liquidity_MW = total_verified_MW * liquidity_share + reserve_buffer
producer_reward_MW = total_verified_MW * reward_share
fulfilled_demand_MW = min(demand_MW, liquidity_MW)
unmet_demand_MW = max(0, demand_MW - liquidity_MW)
slippage_pct = trade_size_MW / (liquidity_MW + epsilon)
```

6. 实验设计

Experiment 1: Agent FDIA Taxonomy

目标：证明 physics-bound verifier 的强项和边界，而不是夸大 100% detection。

攻击类型：

Attack type	生成方式	期望结果
nighttime_impossible	<code>Pmax = 0, Preported > 0</code>	physics bound 应稳定抓住
above_bound	<code>Preported = 1.05-1.80 * Pmax</code>	physics bound 应稳定抓住
near_bound	<code>Preported = 0.98-1.05 * Pmax</code>	tolerance sensitivity
within_bound_fraud	<code>Preported = 0.60-0.95 * Pmax</code>	单一上界不一定能抓住
sensor_drift	residual 持续偏移	adaptive agent 应优于 static
weather_spoofing	irradiance/temp 被篡改	需要 weather consistency
replay_attack	使用其他小时/城市报告	temporal/neighbor

Attack type	生成方式	期望结果
		consistency 更有效
neighbor_inconsistent_attack	单节点与同城邻居显著不一致	neighbor consistency 更有效

比较方法：

```
3-sigma
IQR/MAD
IsolationForest
RandomForest
physics-bound
physics-bound + temporal consistency
physics-bound + temporal + neighbor consistency
adaptive SolarAgent verifier
```

指标：

```
precision
recall
F1
false acceptance rate
false rejection rate
attack-type F1
cross-city transfer F1
```

Experiment 2: Adaptive Agent Verification

目标：回应 EIoT 的 evolutionary 维度。

比较：

```
static physics bound
physics bound + residual memory
physics bound + residual memory + trust score
physics bound + residual memory + trust + neighbor consistency
```

场景：

```
normal weather
cloudy weather: generation = 0.6x
sensor drift: +2% per hour
high noise: measurement noise doubled
cross-city transfer: train/tune on 3 cities, test on 2 cities
```

指标：

```
detection F1
false rejection rate
calibration error
trust convergence speed
records until stable trust
```

Experiment 3: Ratio Sweep and Pareto Frontier

目标：解决 1:3 机制的审稿风险。

扫描：

```
reward_share = [0, 0.10, 0.20, 0.25, 0.30, 0.40, 0.50, 0.60, 0.70, 0.80, 1.00]
```

场景：

```
low demand: 0.5x
normal demand: 1.0x
high demand: 1.5x
cloudy weather: 0.6x generation
high FDIA: 10%, 20%
high volatility: hourly demand variance increased
producer exit risk: low reward causes some agents to stop reporting
```

指标：

```
avg_liquidity_MW
min_daylight_liquidity_MW
avg_slippage_pct
peak_slippage_pct
producer_reward_kWh
reward_gini
fulfilled_demand_ratio
unmet_demand_MWh
liquidity_drawdown_pct
composite_score
pareto_efficient
```

论文表述：

The 1:3 split is not claimed as a universal optimum; it is selected as a robust operating point on the Pareto frontier under the simulated demand, solar-generation, and producer-participation conditions.

Experiment 4: Multi-Agent Coordination Robustness

目标：证明系统不只是 verifier 而是 coordination testbed °

变量：

```
number_of_agents: 50, 250, 500, 1000
FDIA_rate: 0%, 5%, 10%, 20%
demand_level: low, normal, high
weather_condition: normal, cloudy, volatile
planner_policy: strict, balanced, lenient
```

指标：

coordination_success_rate
fulfilled_demand_ratio
average_agent_trust
audit_rate
rejection_rate
market_update_latency_ms
event_log_write_latency_ms

Experiment 5: System Overhead

目标：回应 EIoT 对资源、能耗、延迟约束的关注。

指标：

Layer	Metric
SolarAgent	ms / report
verifier	ms / record
planner policy	ms / decision
market agent	ms / hour-step
event log	ms / event
dataset generation	seconds for 1,200 / 12,000 / 120,000 records
smart contract	gas for registration / set ratio / market update / purchase
frontend	render time for 50 / 250 / 500 nodes

Experiment 6: Connected System Trace

目标：证明 SolarAgents 不是普通 CSV 离线仿真，而是连接当前 FastAPI、frontend 和 on-chain settlement 的 EIoT prototype。

流程：

1. FastAPI starts an agent episode.
2. SolarAgent performs weather perception and generation report.
3. PlannerAgent produces approve/reject/escalate decision.
4. Frontend fetches and displays event trace.
5. Approved reports are aggregated by MarketAgent.
6. FastAPI calls EnergyExchange.updateMarketStep.
7. tx_hash is written back to connected_system_trace.csv.
8. Frontend shows on-chain feedback and updated agent trust.

对比：

offline CSV generation only
offline agent loop
connected agent loop with frontend + local EVM settlement

指标：

end_to_end_latency_ms
frontend_event_fetch_latency_ms
settlement_latency_ms
tx_success_rate
events_with_tx_hash_pct
agent_state_updates_after_feedback
audit_hash_chain_completeness

验收标准：

至少一个 24h connected episode 可以生成 connected_system_trace.csv。
至少一个 approved market step 可以写入 tx_hash。
前端可以展示 selected agent 的 previous_trust -> updated_trust。

7. 论文重构计划

EIoT 2026 要求正文最多 5 页 ’之后 1 页 references °建议不要沿用 9 页 UrbComp 结构 ’而是重写成 workshop paper °

推荐标题

SolarAgents: A Physics-Grounded Embodied IoT Testbed for Verifiable Urban Solar Coordination

备选：

SolarAgents: Lightweight Embodied IoT Agents for Physics-Bounded Smart-Energy Coordination

推荐摘要角度

摘要应避免以 blockchain 开头 °建议以 EIoT 问题开头：

Embodied IoT systems require distributed physical agents to sense, reason, act, and coordinate.

然后再介绍系统：

We present SolarAgents, a physics-grounded EIoT testbed that instantiates urban photovoltaic systems.

三个贡献

建议贡献写成：

1. **Embodied solar-agent abstraction.** We model urban photovoltaic DERs as lightweight embodied IoT agents linking physical body, sensor context, generation reporting, verification, and feedback memory.
2. **Closed-loop verifiable coordination.** We implement an event-driven coordination loop connecting solar agents, planner agents, factory demand agents, market agents, and on-chain settlement.
3. **Reproducible EIoT benchmark.** We release experiments covering FDIA taxonomy, adaptive verification, ratio-sweep market actuation, coordination robustness, and system overhead.

5 页正文结构

1. Introduction (0.6 page)
 - EIoT smart energy challenge
 - why physical grounding + coordination + trust matters
 - contributions
2. Background and Motivation (0.6 page)
 - EIoT/CPD/PICASSO
 - smart-grid FDIA / physics-informed verification
 - why blockchain is only settlement, not main contribution
3. SolarAgents Design (1.2 pages)
 - SolarAgent, FactoryAgent, PlannerAgent, MarketAgent
 - event loop
 - physical bound and feedback memory
 - auditability
4. Implementation (0.7 page)
 - current React/FastAPI/Solidity stack
 - offline benchmark mode and connected demo mode
 - configurable split contract
5. Evaluation (1.3 pages)
 - connected system trace
 - attack taxonomy + baseline comparison
 - adaptive agent verification
 - ratio sweep Pareto
 - system overhead
6. Discussion and Future Work (0.4 page)
 - limits: simulation, lightweight agents, no universal 1:3 optimum
 - future: real DER deployment, edge implementation, stronger communication constraints

8. 最小可行交付版本

如果时间紧，按如下顺序做：

Phase 1: Agent simulation backbone

必须完成：

```
Simulator/agents/types.py
Simulator/agents/solar_agent.py
Simulator/agents/environment.py
Simulator/agents/event_bus.py
Simulator/agents/coordination_loop.py
Simulator/agents/api_bridge.py
Simulator/experiments/eiot_agent_run.py
```

输出：

```
eiot_event_log.csv
audit_logs.csv
```

验收标准：

同一 SolarAgent 在至少 24 个 steps 中保留并更新 trust_score/residual_history。
event log 中能看到 feedback_update 影响下一小时的 decision 或 trust。

Phase 2: Connected demo path

必须完成：

```
Simulator/main.py agent endpoints
client/src/utils/agentApi.js
client/src/components/AgentTrace.jsx
Simulator/data/experiments/connected_system_trace.csv
```

输出：

```
connected_system_trace.csv
frontend Agent Event Trace panel
at least one tx_hash linked to an approved agent action
```

验收标准：

前端可以触发或读取 agent episode。

approved agent step 可以通过 FastAPI 进入 EnergyExchange.updateMarketStep。

event log 中记录 tx_hash, 并能回溯对应 agent_id / planner_action / market_feedback。

Phase 3: Ratio sweep

必须完成：

smart_contract/contracts/EnergyExchange.sol configurable split

smart_contract/test/EnergyExchangeSplit.test.js

Simulator/experiments/ratio_sweep.py

输出：

ratio_sweep_results.csv

ratio_sweep_pareto.png

Phase 4: FDIA taxonomy

必须完成：

Simulator/experiments/attack_taxonomy.py

Simulator/experiments/baseline_detectors.py

输出：

attack_taxonomy_results.csv

baseline_comparison_results.csv

Phase 5: Adaptive verification

必须完成：

```
Simulator/experiments/adaptive_verification.py
```

输出：

```
adaptive_agent_results.csv
```

Phase 6: System overhead

必须完成：

```
Simulator/experiments/system_overhead.py
```

输出：

```
system_overhead_results.csv
```

指标至少包括：

```
agent_step_latency_ms  
verification_latency_ms  
event_log_latency_ms  
settlement_latency_ms  
frontend_fetch_latency_ms
```

Phase 7: Paper figures

建议最终论文图表：

```
Figure 1: EIoT SolarAgents closed-loop architecture  
Figure 2: Connected agent event trace from perception to on-chain settlement  
Figure 3: FDIA taxonomy detection results  
Figure 4: Ratio sweep Pareto frontier  
Table 1: Agent abstraction and EIoT topic mapping  
Table 2: System overhead and scalability
```

9. 录用风险与规避

风险	审稿人可能质疑	规避方式
Agent 只是换名	"These are not embodied agents, only records."	展示 body/perception/action/feedback/memory/event loop
1:3 机制武断	"Why 25/75?"	ratio sweep + Pareto frontier + sensitivity
FDIA 太简单	"Upper-bound attacks are trivial."	attack taxonomy + within-bound/replay/drift + baseline comparison
Blockchain 抢主题	"This is a blockchain paper, not EIoT."	blockchain 只作为 settlement/actuation layer
缺少 evolution	"Where is evolutionary intelligence?"	adaptive residual/trust/calibration experiment
没有真实系统连接	"This is only an offline CSV simulator."	connected demo: FastAPI agent endpoints + frontend trace + tx_hash-linked settlement
Agent 之间无真实依赖	"Agents do not coordinate."	market/factory/planner feedback 必须影响后续 trust/participation/settlement
安全叙事不一致	hardcoded key / CORS *	修工程卫生 ' 并展示 audit log

10. 最终建议

最稳的投稿路线不是增加 GPT API ' 也不是把系统包装成复杂 AI ' 而是 :


```
physics-informed rule agents
+ event-driven coordination
+ adaptive trust/calibration memory
+ persistent auditability
+ connected FastAPI/frontend/on-chain trace
+ ratio-sweep market actuation
+ attack taxonomy benchmark
```

这条路线的优点：

- 符合 EIoT 对 physical grounding 的期待；
- 符合 workshop 对 preliminary/ongoing work 的容忍度；
- 比 LLM agent 更可复现、更可解释；
- 能直接利用当前代码；
- 能用新增实验显著提高论文可信度。
- 能用 connected trace 证明这不是普通 CSV 仿真。

如果只能做最少内容，优先级是：

1. SolarAgent + EventBus + eiot_event_log.csv
2. FastAPI/frontend/on-chain connected trace
3. configurable split + ratio_sweep_results.csv
4. FDIA taxonomy + baseline_comparison_results.csv
5. adaptive trust/calibration experiment
6. persistent audit log + security hygiene

完成这些后，SolarChain 就不再只是 blockchain-backed solar platform，而会更像：

a physics-grounded embodied IoT coordination benchmark for smart-energy agents.

这会明显更贴合 The 9th International Workshop on Embodied Intelligence of Things 的主题。

References

1. EIoT 2026 official website. <https://eiot-workshop.github.io/>
2. CPD 2023 official website. <https://ubicomp-cpd.com/2023.html>
3. PICASSO 2025 official website. <https://picasso-2025.github.io/2025.htm>

4. Yunhao Liu, Xu Wang, Yunhuai Liu, Kebin Liu, Shuai Tong, Jinliang Yuan, and Li Liu. "EIoT: Embodied Intelligence of Things." Tsinghua Science and Technology, 2026. <https://www.sciopen.com/article/10.26599/TST.2025.9010127>
5. Roberto Morabito and Mallik Tatipamula. "The Internet of Physical AI Agents: Interoperability, Longevity, and the Cost of Getting It Wrong." arXiv, 2026. <https://arxiv.org/abs/2603.15900>
6. Shakhrol Iman Siam, Hyunho Ahn, Li Liu, and Mi Zhang. "Artificial Intelligence of Things: A Survey." ACM Transactions on Sensor Networks. <https://dl.acm.org/doi/10.1145/3690639>
7. Maziar Raissi, Paris Perdikaris, and George Em Karniadakis. "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations." Journal of Computational Physics, 2019. <https://ui.adsabs.harvard.edu/abs/2019JCoPh.378..686R/abstract>
8. Mehdi Jabbari Zideh and Sarika Khushalani Solanki. "Physics-Informed Convolutional Autoencoder for Cyber Anomaly Detection in Power Distribution Grids." arXiv, 2023. <https://arxiv.org/html/2312.04758v1>
9. Jannik Zraggen, Yuyan Guo, Antonio Notaristefano, and Lilach Goren Huber. "Physics Informed Deep Learning for Tracker Fault Detection in Photovoltaic Power Plants." PHM Society, 2022. <https://papers.phmsociety.org/index.php/phmconf/article/view/3235>
10. Sara Mohammadi, Frank Eliassen, Yan Zhang, and Hans-Arno Jacobsen. "Detecting False Data Injection Attacks in Peer to Peer Energy Trading Using Machine Learning." IEEE Transactions on Dependable and Secure Computing. <https://www.computer.org/csdl/journal/tq/2022/05/09483661/1vcJui7s3Ys>
11. Haftu Tasew Reda, Adnan Anwar, and Abdun Mahmood. "Comprehensive survey and taxonomies of false data injection attacks in smart grids: attack models, targets, and impacts." Renewable and Sustainable Energy Reviews, 2022. <https://ideas.repec.org/a/eee/rensus/v163y2022ics1364032122003306.html>